

Collecter et nettoyer

Une journée de data scientist : réparer un tableau Pokémon abîmé
exprès en tenant le journal de chaque correction, puis aller chercher des
données soi-même avec une API.

ON REPART AVEC · Un dataset nettoyé et un notebook qui documente chaque correction faite.

01 UNE JOURNÉE DE DATA SCIENTIST 02 CINQ DÉGÂTS 03 SUPPRIMER, REMPLIR OU GARDER
04 LE JOURNAL DES CORRECTIONS 05 UNE API : ALLER CHERCHER SES DONNÉES
06 ENTRE DEUX SÉANCES

À LA FIN DE LA SÉANCE

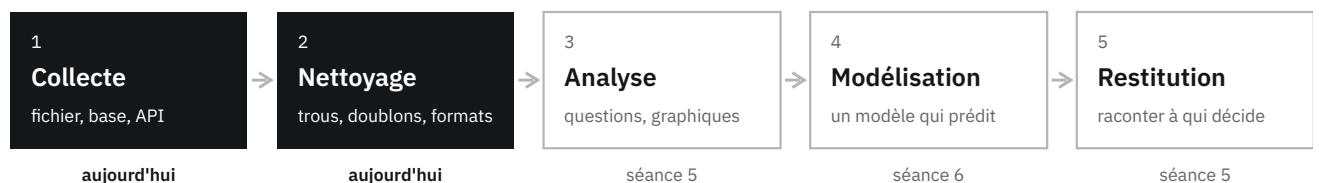
Tu sauras reconnaître les cinq dégâts d'un vrai tableau, doublons, fautes de frappe, nombres en
texte, cases vides, dates mélangées.

Tu sauras réparer chacun en une ou deux lignes de pandas, et décider s'il faut supprimer, remplir ou
garder une case vide.

Tu sauras appeler une API, lire sa réponse JSON et en faire un tableau.

Une journée de data scientist

On imagine des graphiques et des modèles. La réalité commence plus tôt, avec des données sales.



Où passe une journée de travail



Enquête Anaconda 2020, près de 2 400 répondants : 19 % pour charger, plus d'un quart pour nettoyer.

Cinq étapes, et près de la moitié du temps passe dans les deux premières.

– D'OU VIENT LE CHIFFRE DE 45 %

En 2020, l'éditeur Anaconda a interrogé près de 2 400 personnes qui font de la data dans plus de cent pays. En moyenne, elles passent 19 % de leur temps à charger les données et plus d'un quart à les nettoyer, avant de pouvoir s'en servir ; les graphiques prennent environ 21 %, chaque étape du modèle 11 à 12 %.

Le rapport Anaconda 2020 ↗ · un résumé de l'enquête ↗

Cinq dégâts

Le notebook prend les 800 Pokémon propres de la séance 2 et les abîme exprès, comme la vraie vie le ferait. Tout le monde obtient le même tableau abîmé.

df, après abimer(seed=42)

six lignes réelles sur 810

Name	Type 1	HP	date_capture
Palpitoad	water ❷	"75_" ❸	05/03/2024
Charizard	Fier ❷	"78_"	10/05/2024
Charmeleon	fire	NaN ❹	2024-10-17
❶ Growlithe	Fire	"55_"	28 sept 2024 ❺
❶ Growlithe	Fire	"55_"	28 sept 2024
Chandelure	Ghost	"60_"	14 mars 2024 ❺

_ marque un espace invisible à la fin du texte.

- ❶ **doublons**
810 lignes au lieu de 800
10 lignes recopiées, comme Growlithe
- ❷ **fautes de frappe**
56 valeurs de Type 1 au lieu de 18
water, Fire_, Fier : trois « Feu » pour l'ordinateur
- ❸ **nombres en texte**
HP est une colonne de texte
« 75_ » avec un espace : pas de moyenne possible
- ❹ **cases vides**
40 Type 1, 41 HP
NaN, Not a Number : la case a été effacée
- ❺ **dates en trois formats**
255 + 271 + 274
2024-10-17, 05/03/2024, 28 sept 2024

Cinq dégâts, ceux que la vraie vie fait subir à tout tableau.

DÉGÂT	COMMENT LE VOIR	COMMENT LE RÉPARER	SUR LE POKÉMON ABÎMÉ
Doublons	df.duplicated().sum() ()	df.drop_duplicates()	10 lignes en trop
Fautes de frappe	df["Type 1"].unique()	.str.strip().str.capitalize() , puis un dictionnaire des fautes	56 valeurs, puis 30, puis 18
Nombres en texte	df["HP"].dtype affiche object	.str.strip().astype(float)	« 75 » devient 75
Cases vides	df.isna().sum()	fillna , dropna , ou rien	40 Type 1, 40 HP
Dates mélangées	trois formats à l'œil	pd.to_datetime(..., format=..., errors="coerce")	0 date illisible à la fin

Supprimer, remplir ou garder

Une case vide n'est pas toujours un bug : il faut d'abord comprendre pourquoi elle manque.

SUPPRIMER, REMPLIR OU GARDER ? • SUR PAPIER

Pour chaque case vide, choisis ce qu'on en fait.

Supprimer : la case vide rend la ligne inutile (dropna)

Remplir : on met quelque chose à la place de la valeur perdue : la médiane pour un nombre, l'étiquette « Inconnu » pour une catégorie, jamais une valeur devinée (fillna)

Garder : la case vide porte elle-même une information, comme Type 2 pour un Pokémon à un seul type ; on la laisse telle quelle

1. 386 Pokémon sur 800 n'ont rien dans la colonne Type 2.
2. 40 Pokémon ont perdu leurs points de vie (HP).
3. 40 Pokémon ont perdu leur Type 1.
4. Dans Tips, 12 additions n'ont plus de montant (total_bill).
5. Dans Tips, 12 additions ont perdu leur jour de la semaine.
6. Une ligne n'a plus que son nom : toutes les autres cases sont vides.

Les réponses sont dans le corrigé, en fin de leçon.

Le journal des corrections

Si on te demande dans six mois pourquoi il n'y a plus que 800 lignes, tu dois pouvoir répondre. Le notebook tient une simple liste, remplie au fil des réparations.

```
journal = []  
def noter(message):  
    journal.append(message)
```

1. le journal, une ligne par réparation

ce que le notebook affiche à la fin, dans l'ordre

1. Départ : 810 lignes, 471 cases vides
2. Doublons : 10 lignes identiques supprimées, il reste 800 lignes
3. Type 1 : espaces et majuscules normalisés, 12 fautes corrigées → 18 types valides
4. HP : texte converti en nombre (strip + astype)
5. HP : 40 cases vides remplies par la médiane (65)
6. Type 1 : cases vides remplacées par 'Inconnu'
7. date_capture : 3 formats convertis en vraies dates (to_datetime)

2. la preuve

le tableau réparé face au tableau propre

	propre	réparé
lignes	800	800
types de Type 1	18	19 dont « Inconnu »
HP moyen	69,3	69,1

L'écart de HP vient des 40 cases remplies par 65 : le journal dit pourquoi le chiffre a bougé.

Chaque réparation s'écrit ; le tableau réparé se compare au tableau d'origine.

Une API : aller chercher ses données

Une API est une porte par laquelle un programme demande des données à un site : au restaurant, tu ne vas pas en cuisine, tu passes commande.

1. la commande : une URL

`pokeapi.co/api/v2/pokemon/pikachu`

`requests.get(url)` passe la commande,
`.json()` ouvre le plat

2. le plat : du JSON, des dictionnaires dans des listes

```
{
  "name": "pikachu",
  "height": 4,
  "weight": 60,
  "types": [ { "type": { "name": "electric" } } ],
  "stats": [ { "stat": { "name": "hp" }, "base_stat": 35 },
              { "stat": { "name": "attack" }, "base_stat": 55 }, ... ],
  ... 16 autres clés
}
```

le type est rangé trois niveaux plus bas :

`data["types"][0]["type"]["name"] → "electric"`

3. une ligne de DataFrame

un appel par Pokémon, une ligne chacun

nom	type	taille	poids	hp	attack	defense	speed
pikachu	electric	4	60	35	55	40	90

Une API répond en JSON ; on y va chercher ce qu'il faut pour remplir une ligne de tableau.

- ET SI LE RÉSEAU TOMBE ?

Un appel réseau peut échouer : pas de connexion, site en panne, Pokémon mal orthographié. La fonction `demande` du notebook enveloppe l'appel dans un `try/except` et renvoie une réponse de secours écrite à l'avance, pour que la suite du notebook tourne quand même. La deuxième API du notebook, Open-Meteo, donne la météo d'une latitude et d'une longitude, gratuitement et sans clé.

Dans la PokéAPI, `height` est en décimètres et `weight` en hectogrammes : Pikachu mesure 0,4 m et pèse 6 kg. Lire la documentation d'une API, c'est aussi apprendre ses unités.

À toi : nettoie Tips

Tips, ce sont 244 additions d'un restaurant américain. Abîmées par la même fonction, elles deviennent 254 lignes, avec 14 façons d'écrire les 4 jours de la semaine.

Répare dans l'ordre : doublons, catégories, nombres, cases vides, dates, une ligne de journal par correction.

Justifie tes cases vides dans le journal : une addition sans montant, un jour effacé, que fais-tu de chacun ?

Prouve ton travail en comparant au tableau propre, puis sauvegarde `tips_propre.csv` : c'est ton livrable.

Entre deux séances

Le carnet `04_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées. Il reprend la réparation du Pokémon étape par étape, une API avec son plan B, et finit par Tips de A à Z.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Dans le tableau abîmé, « fire », « Fire » suivi d'un espace, et « Fire » comptent comme trois types différents. Quelle ligne en fait un seul ?
 - A. `df["Type 1"].str.strip().str.capitalize()`
 - B. `df["Type 1"].dropna()`
 - C. `df.drop_duplicates()`
 - D. `df["Type 1"].astype(float)`
2. Pour convertir la colonne HP, le notebook écrit `astype(float)` et pas `astype(int)`. Pourquoi ?
 - A. Les points de vie ont des virgules
 - B. Il reste des cases vides, et NaN ne rentre pas dans une colonne d'entiers
 - C. float est plus rapide
 - D. int ne sait pas lire le texte

3. Après réparation, le HP moyen vaut 69,1 contre 69,3 dans le tableau propre. Qu'est-ce que ça veut dire ?
- A. Le nettoyage a raté
 - B. Il manque des Pokémon
 - C. Les 40 cases remplies par 65 ne valent pas les vraies valeurs effacées : la moyenne bouge un peu
 - D. pandas arrondit mal
4. La PokéAPI renvoie data, un JSON. Quelle ligne donne le type de Pikachu ?
- A. data["type"]
 - B. data["types"]["name"]
 - C. data["types"][0]["type"]["name"]
 - D. data.type

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Un vrai tableau est sale ; le préparer prend près de la moitié du temps du métier.
2. Le bilan de santé d'abord, info, isna, duplicated, unique, avant de toucher à quoi que ce soit.
3. Chaque dégât a son outil, drop_duplicates, strip et capitalize, astype, fillna ou dropna, to_datetime.
4. Une case vide se supprime, se remplit ou se garde, et le choix se justifie dans le journal.
5. Une API, c'est une URL, une réponse JSON, puis un tableau ; toujours avec un plan B si le réseau tombe.

Corrigé

Supprimer, remplir ou garder ?

1. **Garder** · Ce n'est pas un trou : ces Pokémon n'ont qu'un seul type. Le vide porte une information, le remplir l'effacerait.
2. **Remplir** · Le reste de la ligne est précieux. La médiane des HP, 65, est une valeur raisonnable qui ne dépend pas des rares géants, jusqu'à 255.
3. **Remplir** · Ici le vide ne dit rien en soi : le type a été effacé. La ligne reste utile, mais on ne devine pas son type : la case reçoit l'étiquette « Inconnu ». Écrire « Water » au hasard serait mentir.
4. **Supprimer** · Une addition sans montant ne sert à rien pour étudier les pourboires. 12 lignes sur 244, on les retire et on l'écrit dans le journal.
5. **Remplir** · Le jour a été effacé, le vide ne dit rien en soi. Le montant et le pourboire restent utiles : le jour reçoit l'étiquette « Inconnu », et chaque calcul par jour le montrera à part.
6. **Supprimer** · Il n'y a plus rien à analyser, et remplir toutes ces cases reviendrait à fabriquer un Pokémon. Une ligne vide va à la corbeille, avec sa ligne de journal.

Mini-quiz

1. **A. `df["Type 1"].str.strip().str.capitalize()`** · `strip` enlève l'espace invisible au bout, `capitalize` remet une majuscule devant et des minuscules derrière. Le tableau passe de 56 valeurs à 30 ; les lettres inversées comme « Fier » demandent ensuite un dictionnaire de corrections.
2. **B. Il reste des cases vides, et NaN ne rentre pas dans une colonne d'entiers** · NaN est lui-même un nombre à virgule : une colonne d'entiers ne peut pas le contenir. On passe par float, on remplit les vides, et seulement ensuite on repasse en int.
3. **C. Les 40 cases remplies par 65 ne valent pas les vraies valeurs effacées : la moyenne bouge un peu** · Remplir, c'est remplacer une valeur perdue par une valeur raisonnable, pas par la vraie. Un petit écart est normal ; ce qui compte, c'est que le journal dise d'où il vient.
4. **C. `data["types"][0]["type"]["name"]`** · `types` est une liste (un Pokémon peut en avoir deux), d'où le `[0]` ; chaque élément est un dictionnaire qui contient `type`, lui-même un dictionnaire qui contient `name`. On descend niveau par niveau jusqu'à « electric ».